

# Projeto e Análise de Algoritmos

Algoritmos de busca

Thiago Pinheiro de Araújo

EMAp/FGV

2026.2

Apresentar **algoritmos de busca**, discutir sua complexidade, estruturas de dados utilizadas e exemplos de implementação.

# Busca em vetor ordenado

# Busca em vetor ordenado

## O problema

- Dado um vetor ordenado de números inteiros

```
int v = {2, 5, 9, 18, 23, 27, 32, 33, 37, 41, 43, 45};
```

- Escreva um algoritmo que :
  - Receba como parâmetro um vetor ( $v$ ) e um número ( $x$ ) a ser buscado.
  - Retorne o índice do vetor no qual se encontra o número.
- Caso o número não esteja presente no vetor, retorne  $-1$ ;
- Assuma que o vetor não possui números repetidos.

## Busca sequencial

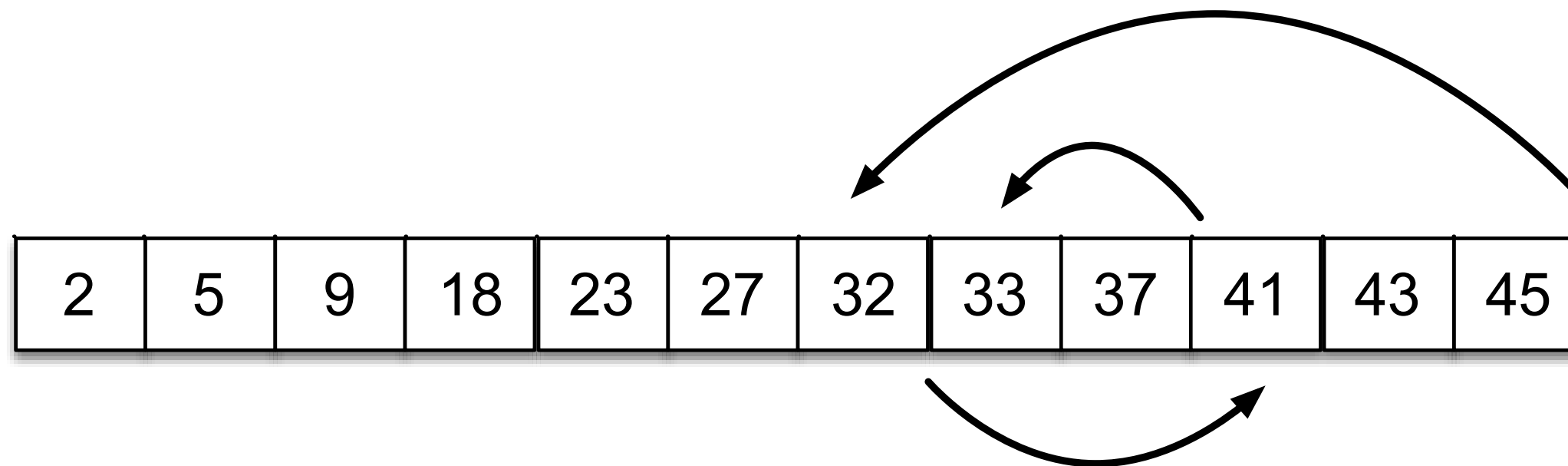
- É possível escrever esse algoritmo trivialmente percorrendo os elementos do vetor verificando para cada um deles se o número corrente é igual ao número de interesse.

```
int search(const int v[], int size, int x) {  
    for (int i=0; i < size; i++) {  
        if (v[i] == x) {  
            return i;  
        }  
    }  
    return -1;  
}
```

- Considerando o pior caso esse algoritmo apresenta complexidade  $\Theta(n)$ .

## Busca binária

- Considerando que o vetor de entrada está ordenado, podemos escrever um algoritmo mais eficiente chamado de **busca binária**.



- O algoritmo consiste em avaliar se o elemento buscado ( $x$ ) é o elemento no meio do vetor ( $m$ ), e caso não seja executar a mesma operação sucessivamente para a metade superior (caso  $x > m$ ) ou inferior (caso  $x < m$ ).

## Busca binária

- Exemplo de uma implementação recursiva:

```
int search(int v[], int leftInx, int rightInx, int x) {
    int midInx = (leftInx + rightInx) / 2;
    int midValue = v[midInx];
    if (midValue == x) {
        return midInx;
    }
    if (leftInx >= rightInx) {
        return -1;
    }
    if (x > midValue) {
        return search(v, midInx + 1, rightInx, x);
    } else {
        return search(v, leftInx, midInx - 1, x);
    }
}
```

## Busca binária

- Exemplo de uma implementação iterativa:

```
int search(const int v[], int size, int x) {
    int leftInx = 0;
    int rightInx = size - 1;
    while (leftInx <= rightInx) {
        int midInx = (leftInx + rightInx) / 2;
        int midValue = v[midInx];
        if (x == midValue) {
            return midInx;
        }
        if (x < midValue) {
            rightInx = midInx - 1;
        } else {
            leftInx = midInx + 1;
        }
    }
    return -1;
}
```

- Podemos escrever a função de complexidade como:

$$T(n) \leq T\left(\frac{n}{2}\right) + c$$

- Ou seja:

$$\begin{aligned} T(n) &\leq T\left(\frac{n}{2}\right) + c \\ &\leq T\left(\frac{n}{4}\right) + c + c \\ &\leq T\left(\frac{n}{2^k}\right) + kc \end{aligned}$$

$$\begin{aligned} T(1) &= 1 \\ \frac{n}{2^k} &= 1 \\ k &= \log(n) \end{aligned}$$

$$\begin{aligned} T(n) &\leq T\left(\frac{n}{2^k}\right) + kc \\ &\leq T(1) + c \log(n) \\ &\leq c \log(n) \end{aligned}$$

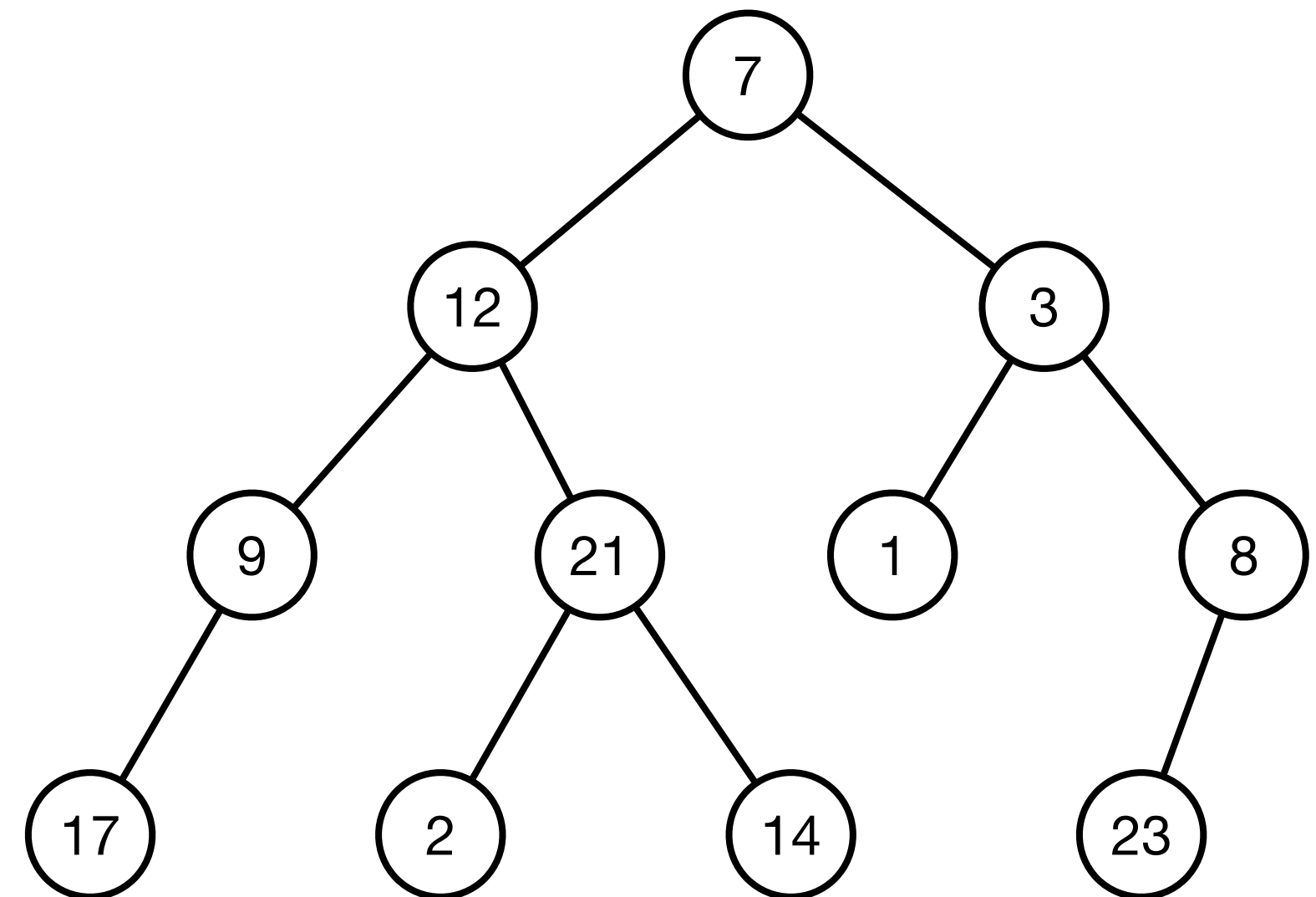
- Portanto, a complexidade da busca binária é  $O(\log(n))$

# Árvores binárias

# Árvores binárias

## Definição

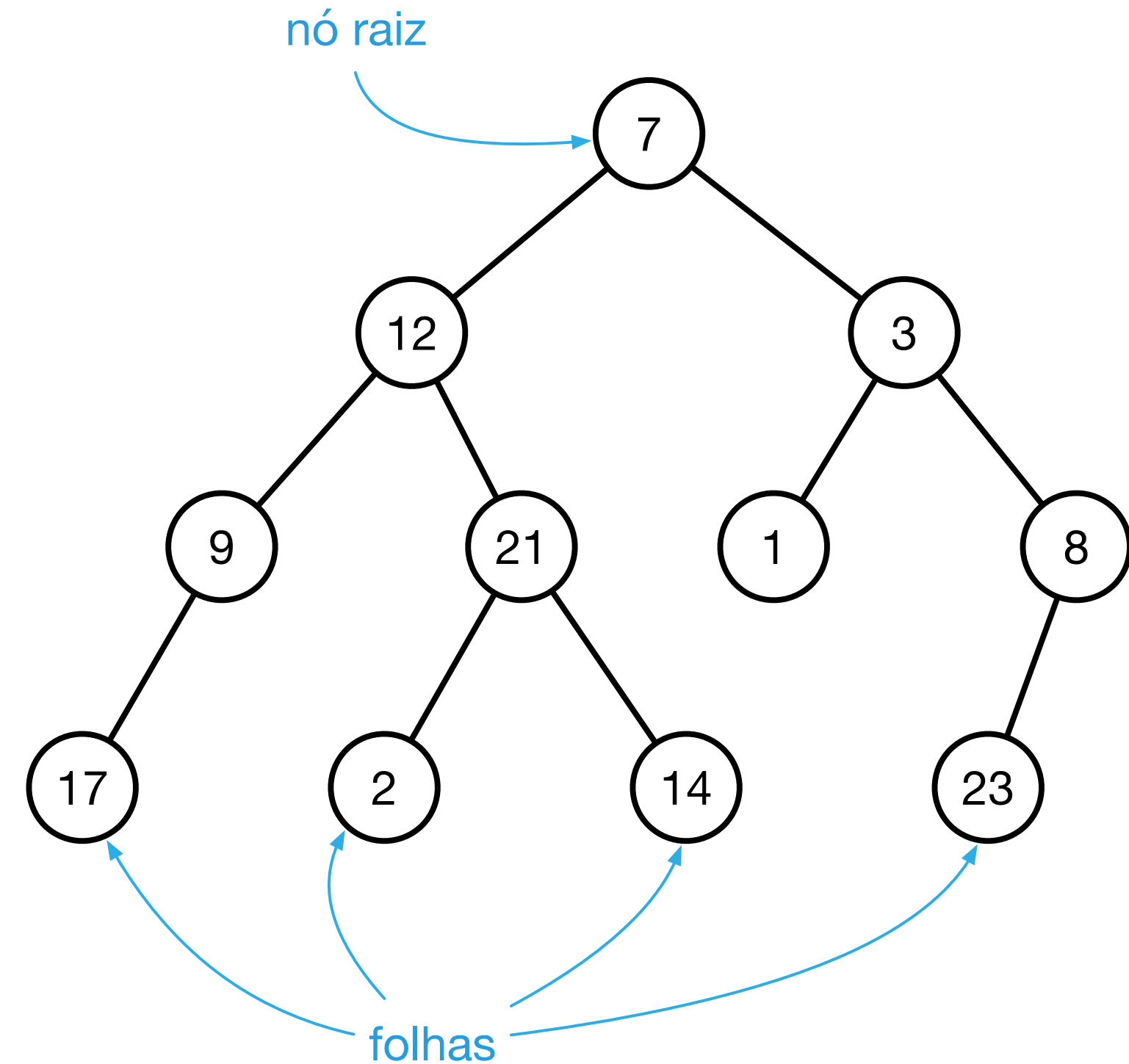
- Uma **árvore binária** consiste em uma estrutura de dados capaz de armazenar um conjunto de nós.
- Todo nó possui uma chave
  - Opcionalmente um valor (dependendo da aplicação).
- Cada nó possui referências para dois filhos
  - Sub-árvores da direita e da esquerda.
- Toda sub-árvore também é uma árvore.



# Árvores binárias

## Definição

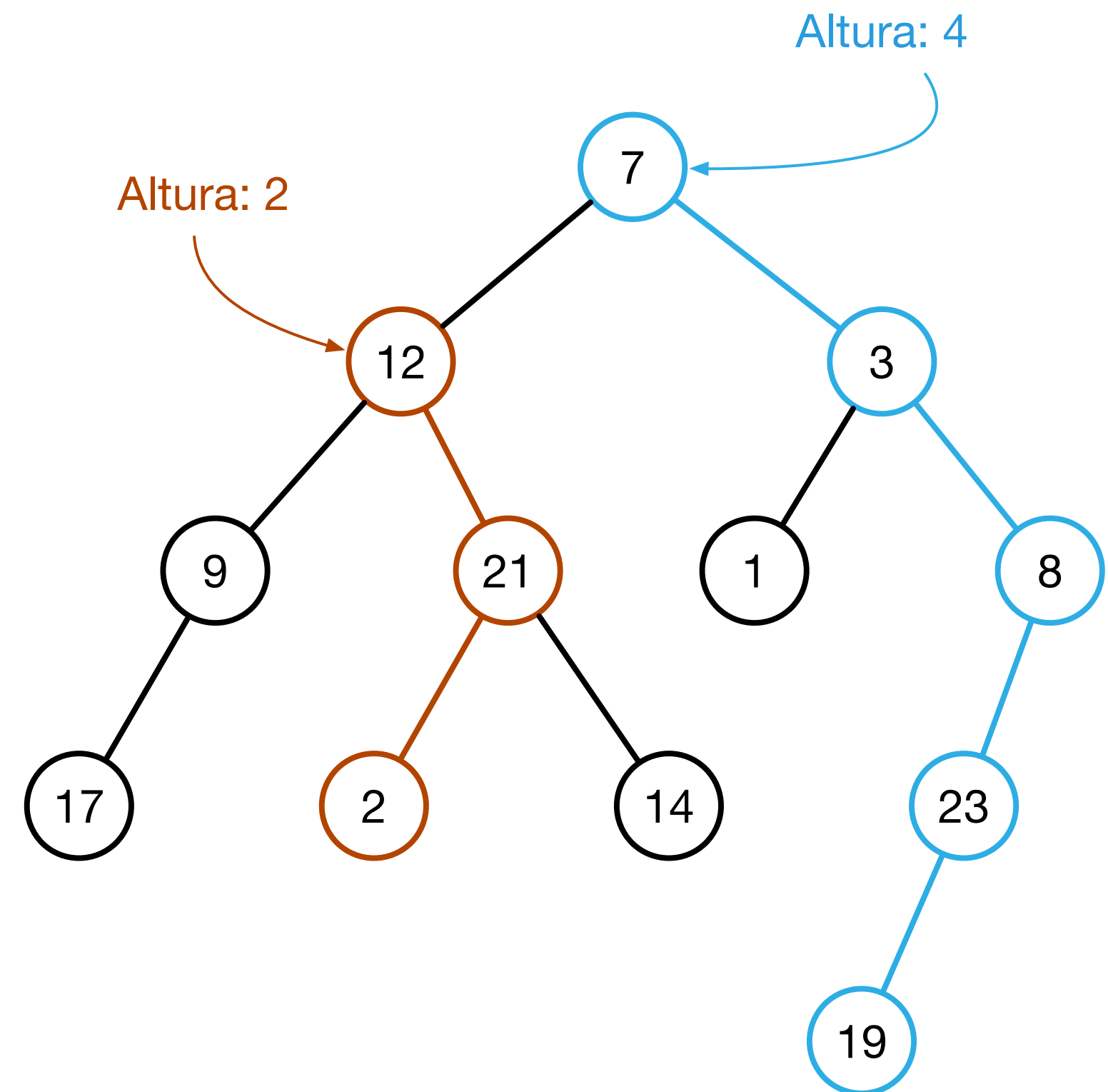
- O nó pode opcionalmente conter um ponteiro para o seu nó pai
  - Depende dos requisitos do algoritmo.
- Um nó que não possui pai é chamado de raiz da árvore.
- Caso o nó não possua filhos ele é considerado uma folha.



# Árvores binárias

## Definição

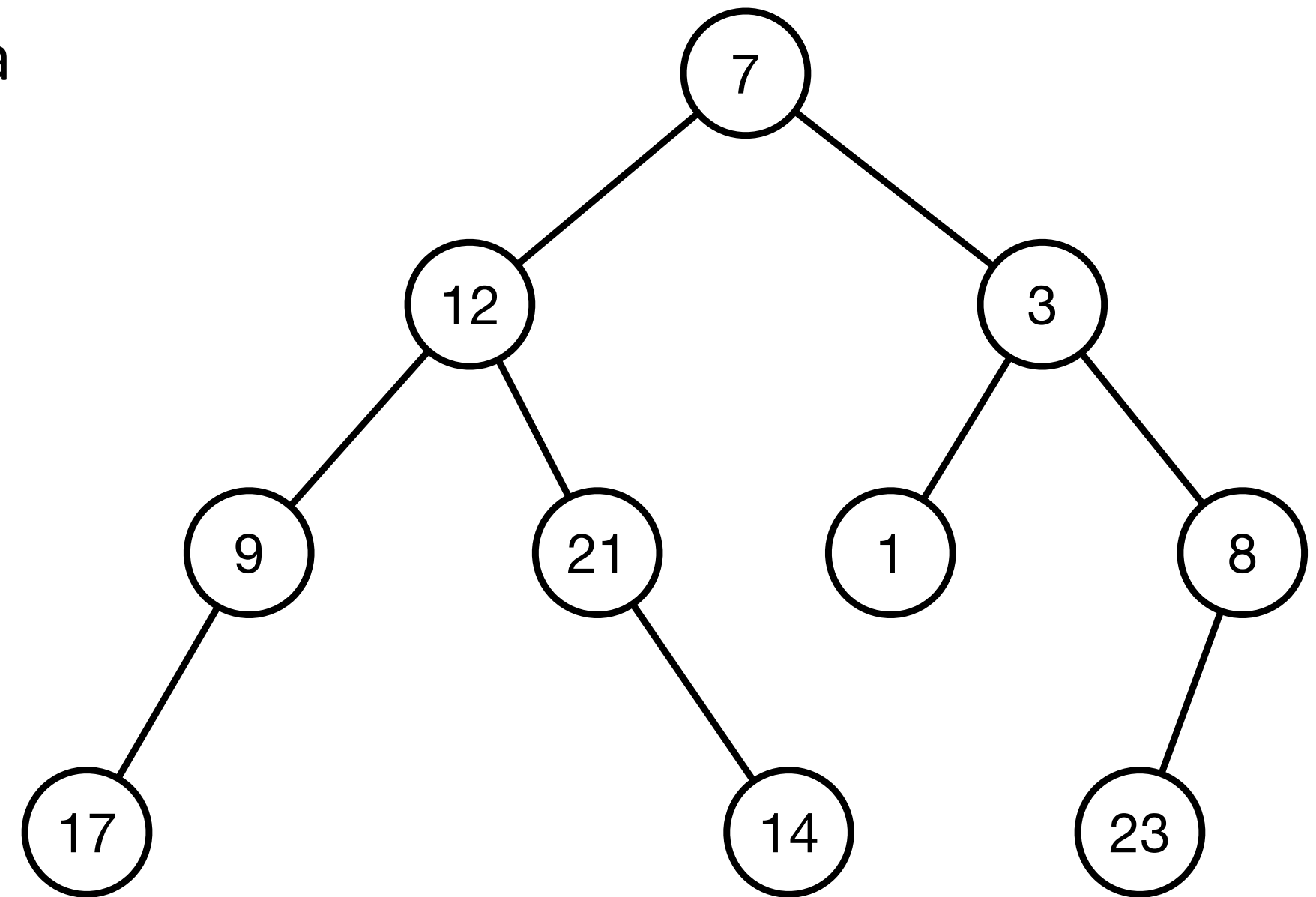
- A altura de um nó consiste na distância entre ele e a folha mais afastada.
- Portanto, a altura de uma árvore consiste na altura da sua raiz
  - Uma árvore com um único nó tem altura 0.
- Dada uma árvore com altura  $h$ , quais os números máximo e mínimo de nós que ela pode ter?
  - $n_{min} = h + 1$
  - $n_{max} = 2^{h+1} - 1$



# Árvores binárias

## Definição

- Uma árvore está **balanceada** caso a altura das sub-árvores de cada nó apresente uma diferença de no máximo 1.
- Como poderíamos determinar a altura  $h$  de uma árvore balanceada em função do número de nós  $n$ ?
  - $h = \log(n)$



# Árvores binárias

## Estrutura de dados

- Uma implementação simplificada da estrutura de dados seria:
- Utilizada da seguinte forma:

```
// Create nodes
Node * n1 = new Node(1, 'a');
Node * n2 = new Node(2, 'b');
Node * n3 = new Node(3, 'c');
Node * n4 = new Node(4, 'd');
Node * n5 = new Node(5, 'e');
Node * n6 = new Node(6, 'f');

// Build the tree
n1->setLeftNode(n2);
n1->setRightNode(n3);
n2->setLeftNode(n4);
n2->setRightNode(n5);
n3->setRightNode(n6);

// Set the root
Node * rootNode = n1;
```

```
class Node {
public:
    Node(int key, char data)
        : m_key(key)
        , m_data(data)
        , m_leftNode(nullptr)
        , m_rightNode(nullptr)
        , m_parentNode(nullptr) {}

    Node & leftNode() const { return * m_leftNode; }
    void setLeftNode(Node * node) { m_leftNode = node; }

    Node & rightNode() const { return * m_rightNode; }
    void setRightNode(Node * node) { m_rightNode =
node; }

    Node & parentNode() const { return * m_parentNode; }
    void setParentNode(Node * node) { m_parentNode =
node; }

private:
    int m_key;
    char m_data;
    Node * m_leftNode;
    Node * m_rightNode;
    Node * m_parentNode;
};
```

# Árvore binária de busca

## Altura de uma árvore

- **Exercício:** dada uma árvore binária  $A$  com  $n$  nós encontre a sua altura.

- Solução:

```
int nodeHeight(Node * node) {
    if (node == nullptr) {
        return -1;
    }

    int leftHeight = nodeHeight(node->leftNode());
    int rightHeight = nodeHeight(node->rightNode());
    if (leftHeight < rightHeight) {
        return rightHeight + 1;
    } else {
        return leftHeight + 1;
    }
}
```

- Complexidade:  $\Theta(n)$

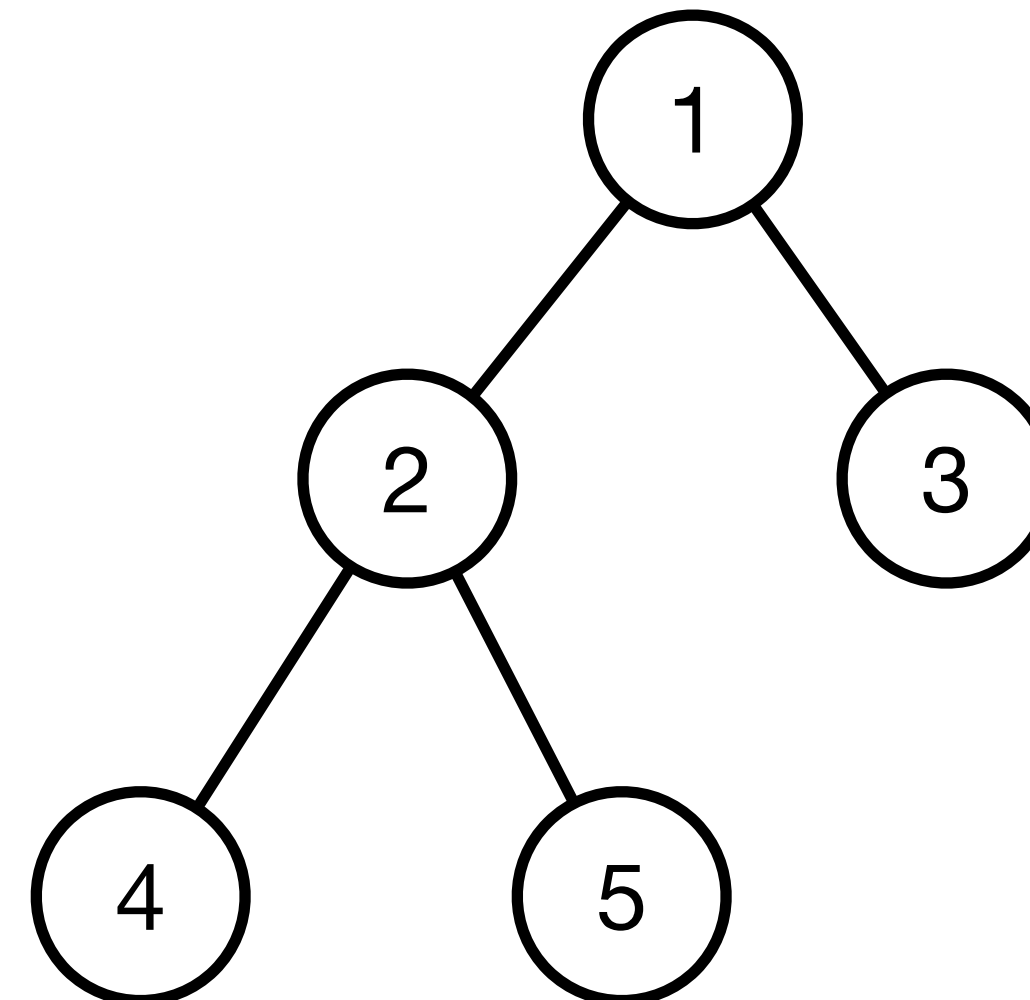
- Diferente de estruturas de dados como vetores, listas, filas e pilhas, as árvores possuem múltiplas formas de serem percorridas:

- **Busca em profundidade (DFS):**

- Em ordem (esquerda-raiz-direita)
  - [4, 2, 5, 1, 3]
- Pré-ordem (raiz-esquerda-direita)
  - [1, 2, 4, 5, 3]
- Pós-ordem (esquerda-direita-raiz)
  - [4, 5, 2, 3, 1]

- **Busca em largura (BFS):**

- [1, 2, 3, 4, 5]



- **Problema:** dada uma árvore binária  $A$  imprima a chave de todos os nós através da busca em profundidade (em ordem).
- Solução:

```
void printTreeDFSInorder(class Node * node) {  
    if (node == nullptr) {  
        return;  
    }  
    printTreeDFSInorder(node->leftNode());  
    cout << node->key() << " ";  
    printTreeDFSInorder(node->rightNode());  
}
```

- **Problema:** dada uma árvore binária  $A$  imprima a chave de todos os nós através da busca em profundidade (pré-ordem).
- Solução:

```
void printTreeDFSPreorder(class Node * node) {  
    if (node == nullptr) {  
        return;  
    }  
    cout << node->key() << " ";  
    printTreeDFSPreorder(node->leftNode());  
    printTreeDFSPreorder(node->rightNode());  
}
```

- **Exercício:** dada uma árvore binária  $A$  imprima a chave de todos os nós através da busca em profundidade (pós-ordem).
- Solução:

```
void printTreeDFSPostorder(class Node * node) {  
    if (node == nullptr) {  
        return;  
    }  
    printTreeDFSPostorder(node->leftNode());  
    printTreeDFSPostorder(node->rightNode());  
    cout << node->key() << " ";  
}
```

- **Problema:** dada uma árvore binária  $A$  imprima a chave de todos os nós através da busca em largura.
- Solução (ineficiente):

```
void printTreeBFS(Node * root) {  
    int depth = nodeHeight(root) + 1;  
    for (int i=0; i < depth; i++)  
        printLevel(root, i + 1);  
}
```

```
void printLevel(Node * root, int level) {  
    if (root == nullptr) {  
        return;  
    }  
  
    if (level == 1) {  
        cout << root->key() << " ";  
    } else if (level > 1) {  
        printLevel(root->leftNode(), level - 1);  
        printLevel(root->rightNode(), level - 1);  
    }  
}
```

- Complexidade:  $\Theta(n^2)$

- **Problema:** dada uma árvore binária  $A$  imprima a chave de todos os nós através da busca em largura.

- Solução (eficiente):

```
void printTreeBFSWithQueue (Node * root) {  
    if (root == nullptr) {  
        return;  
    }  
    queue<Node*> queue;  
    queue.push(root);  
    while (!queue.empty()) {  
        Node * node = queue.front();  
        cout << node->key() << " ";  
        queue.pop();  
        Node * childNode = node->leftNode();  
        if (childNode) {  
            queue.push(childNode);  
        }  
        childNode = node->rightNode();  
        if (childNode) {  
            queue.push(childNode);  
        }  
    }  
}
```

- Complexidade:  $\Theta(n)$

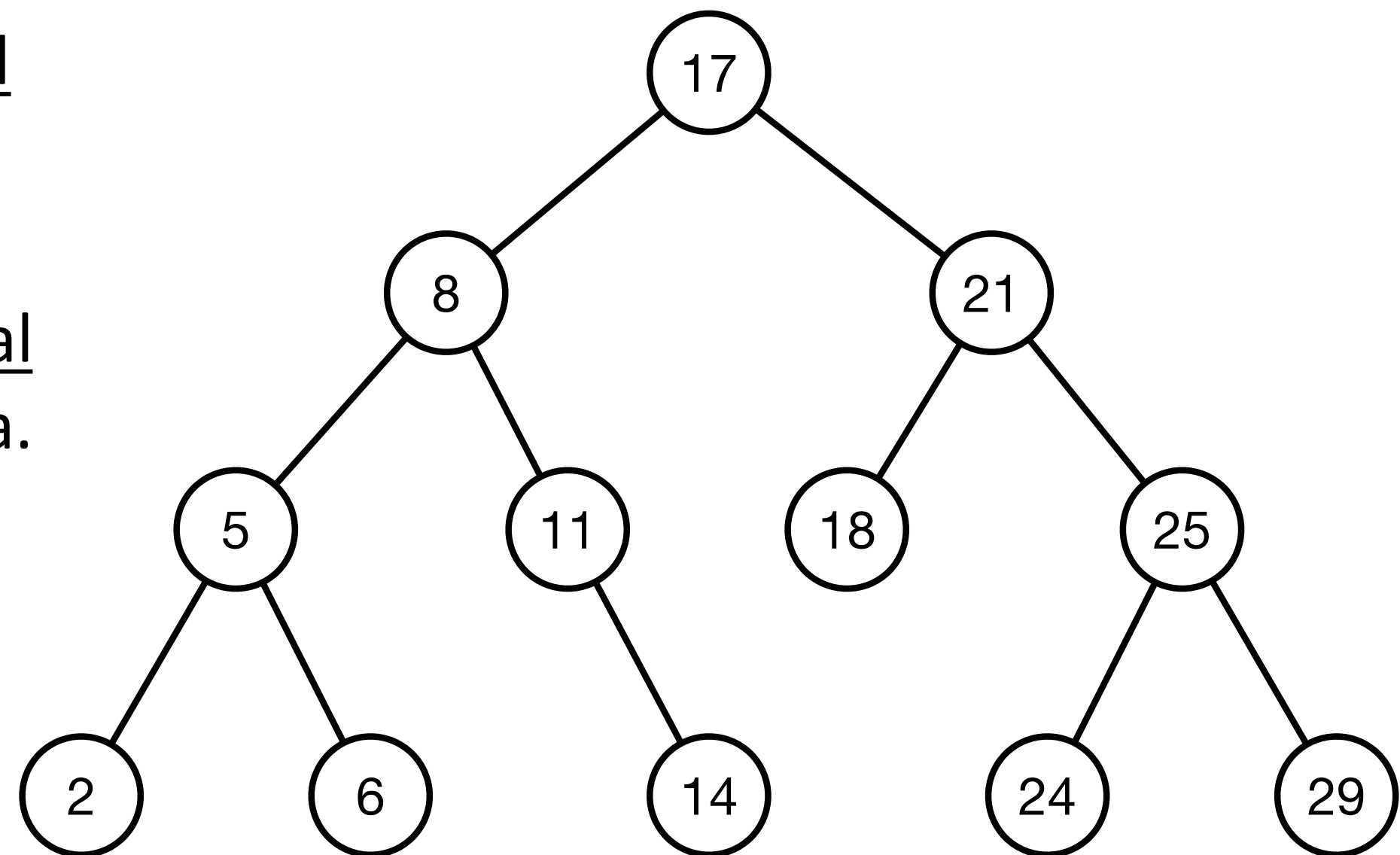
# Árvores binárias de busca

# Árvore binária de busca

## Definição

- Uma **árvore binária de busca (BST)** é uma árvore binária que respeita as seguintes propriedades:
  - A chave de cada nó é maior ou igual a chave da raiz da sub-árvore esquerda.
  - A chave de cada nó é menor ou igual a chave da raiz da sub-árvore direita.

$$left.key \leq key \leq right.key$$



# Árvore binária de busca

## Algoritmos de pesquisa

- **Problema:** dada uma árvore binária de busca  $A$  com altura  $h$  encontre o nó cuja chave seja  $k$ .
- Solução (recursiva):

```
Node * binaryTreeSearchRecursive(Node * node, int key) {  
    if (node == nullptr || node->key() == key) {  
        return node;  
    }  
    if (node->key() > key) {  
        return binaryTreeSearchRecursive(node->leftNode(), key);  
    } else {  
        return binaryTreeSearchRecursive(node->rightNode(), key);  
    }  
}
```

- Complexidade:  $\Theta(h)$

# Árvore binária de busca

## Algoritmos de pesquisa

- **Problema:** dada uma árvore binária de busca  $A$  com altura  $h$  encontre o nó cuja chave seja  $k$ .
- Solução (iterativa):

```
Node * binaryTreeSearchIterative(Node * node, int key) {  
    while (node != nullptr && node->key() != key) {  
        if (node->key() > key) {  
            node = node->leftNode();  
        } else {  
            node = node->rightNode();  
        }  
    }  
    return node;  
}
```

## Exercícios

- **Exercício 1:** dada uma árvore binária de busca  $A$  com altura  $h$  encontre o índice do nó com índice mínimo.
- **Exercício 2:** dada uma árvore binária de busca  $A$  com altura  $h$  encontre o índice do nó com índice máximo.
- **Exercício 3:** dado um nó  $n_1$  de uma árvore binária de busca  $A$  com altura  $h$  encontre o nó  $n_2$  cujo índice é o sucessor da sequência ordenada.
- **Exercício 4:** dado um nó  $n_1$  de uma árvore binária de busca  $A$  com altura  $h$  encontre o nó  $n_2$  cujo índice é o predecessor da sequência ordenada.
- **Exercício 5:** dada uma árvore binária de busca  $A$  com altura  $h$  e uma chave  $k$  crie um novo nó capaz de inserir a chave mantendo as propriedades de BST.
- **Exercício 6:** dada uma árvore binária de busca  $A$  com altura  $h$  e uma chave  $k$  remova o nó com esta chave mantendo as propriedades de BST.

# Árvore binária de busca

## Algoritmos de pesquisa

- **Exercício 1:** dada uma árvore binária de busca  $A$  com altura  $h$  encontre o índice do nó com índice mínimo.

- Solução:

```
Node * binaryTreeSearchMin(Node * node) {  
    if (node == nullptr) {  
        return node;  
    }  
    while (node->leftNode() != nullptr) {  
        node = node->leftNode();  
    }  
    return node;  
}
```

- Complexidade:  $\Theta(h)$

# Árvore binária de busca

## Algoritmos de pesquisa

- **Exercício 2:** dada uma árvore binária de busca  $A$  com altura  $h$  encontre o índice do nó com índice máximo.

- Solução:

```
Node * binaryTreeSearchMax(Node * node) {  
    if (node == nullptr) {  
        return node;  
    }  
    while (node->rightNode() != nullptr) {  
        node = node->rightNode();  
    }  
    return node;  
}
```

- Complexidade:  $\Theta(h)$

# Árvore binária de busca

## Algoritmos de pesquisa

- **Exercício 3:** dado um nó  $n_1$  de uma árvore binária de busca  $A$  com altura  $h$  encontre o nó  $n_2$  cujo índice é o sucessor da sequência ordenada.

- Solução:

```
Node * binaryTreeSearchSuccessor(Node * node) {  
    if (node == nullptr) {  
        return node;  
    }  
    if (node->rightNode() != nullptr) {  
        return binaryTreeSearchMin(node->rightNode());  
    }  
    Node * parentNode = node->parentNode();  
    while (parentNode != nullptr && node == parentNode->rightNode()) {  
        node = parentNode;  
        parentNode = parentNode->parentNode();  
    }  
    return parentNode;  
}
```

- Complexidade:  $\Theta(h)$

- **Exercício 4:** dado um nó  $n_1$  de uma árvore binária de busca  $A$  com altura  $h$  encontre o nó  $n_2$  cujo índice é o predecessor da sequência ordenada.

- Solução:

```
Node * binaryTreeSearchPredecessor(Node * node) {  
    if (node == nullptr) {  
        return node;  
    }  
    if (node->leftNode() != nullptr) {  
        return binaryTreeSearchMax(node->leftNode());  
    }  
    Node * parentNode = node->parentNode();  
    while (parentNode != nullptr && node == parentNode->leftNode()) {  
        node = parentNode;  
        parentNode = parentNode->parentNode();  
    }  
    return parentNode;  
}
```

- Complexidade:  $\Theta(h)$

## Algoritmo de inserção

- **Exercício 5:** dada uma árvore binária de busca  $A$  com altura  $h$  e uma chave  $k$  crie um novo nó capaz de inserir a chave mantendo as propriedades de BST.

- Solução:

```
Node * binaryTreeInsert(Node * root, int key) {  
    if (root == nullptr) {  
        return new Node(key);  
    }  
    if (key < root->key()) {  
        root->setLeftNode(binaryTreeInsert(root->leftNode(), key));  
    } else {  
        root->setRightNode(binaryTreeInsert(root->rightNode(), key));  
    }  
    return root;  
}
```

- Complexidade:  $\Theta(h)$

## Algoritmo de remoção

- **Exercício 6:** dada uma árvore binária de busca  $A$  com altura  $h$  e uma chave  $k$  remova o nó com esta chave mantendo as propriedades de BST.

- Solução:

```
Node * binaryTreeDelete(Node * root, int key)
{
    if (root == nullptr) {
        return root;
    }
    if (key < root->key()) {
        root->setLeftNode(binaryTreeDelete(root->leftNode(), key));
    } else if (key > root->key()) {
        root->setRightNode(binaryTreeDelete(root->rightNode(), key));
    } else {
        root = binaryTreeDeleteNode(root);
    }
    return root;
}
```

- Continua no próximo slide...

# Árvore binária de busca

## Algoritmo de remoção

- **Exercício 6:** dada uma árvore binária de busca  $A$  com altura  $h$  e uma chave  $k$  remova o nó com esta chave mantendo as propriedades de BST.

- Solução:

```
Node * binaryTreeDeleteNode(Node * root) {
    if (root->leftNode() == nullptr && root->rightNode() == nullptr) {
        delete root;
        root = nullptr;
    } else if (root->leftNode() == nullptr) {
        Node * newRoot = root->rightNode();
        delete root;
        root = newRoot;
    }
    else if (root->rightNode() == nullptr) {
        Node * newRoot = root->leftNode();
        delete root;
        root = newRoot;
    }
    else {
        Node * newRoot = binaryTreeSearchMin(root->rightNode());
        root->setKey(newRoot->key());
        root->setRightNode(binaryTreeDelete(root->rightNode(), newRoot->key()));
    }
    return root;
}
```

- Complexidade:  $\Theta(h)$

# Perguntas?

`thiago.p.araujo@fgv.br`